APRESENTAÇÃO FINAL

TUPÃ

Programe o ESP32 em português.



Michel Lutegar · André Costa
Felipe Dutrain · Guilherme Lopes

Orientador: Prof. William Barbosa

LINGUAGENS FORMAIS E COMPILADORES · IBMEC

O PROBLEMA

O mesmo teste, em duas linguagens.

firmware/teste_hardware.ino — 51 linhas

```
#define SS_PIN   5
#define RST_PIN  22
#define BTN      4
// ...
void setup() {
  pinMode(BTN, INPUT_PULLUP);
  // ...
  rfid.PCD_Init();
  delay(50);
  // ...
void loop() {
  int b = digitalRead(BTN);
  int ir = digitalRead(IR_PIN);
  if (b != lastBtn) {
  // ...
    lastBtn = b;
  }
  // ...
  delay(20);
}
```

examples/teste_hardware.tupa — 23 linhas

```
dispositivo teste_hardware {
  ao iniciar {
    escrever("== Teste de hardware Tupa ==");
    escrever("LED verde...");
    led.verde();
    esperar(700);
    escrever("LED vermelho...");
    led.vermelho();
    esperar(700);
    escrever("Aperte o botao, passe a mao no IR e aproxime um cartao.");
  }
  ao botao(1) {
    led.vermelho();
    esperar(300);
  }
  ao obstaculo {
  }
  ao cartao {
    escrever("Cartao UID: " + cartao.uid);
    led.verde();
    esperar(800);
  }
}
```

A LACUNA

Ninguém une português, texto e placa real.

Arduino IDE C++ e pinos.	**MakeCode, Blockly** Só blocos.	**Flipper Zero** Produto pronto.	**Portugol Studio** Sem placa.

Tupã texto em português, firmware de verdade.

A LINGUAGEM

Legível, por evento, sem pino à vista.

examples/controle_nfc.tupa — 12 linhas

```
dispositivo controle {

    inteiro tentativas = 3;

    ao botao(1) {

        # le um cartao NFC
        cartao = nfc.ler();

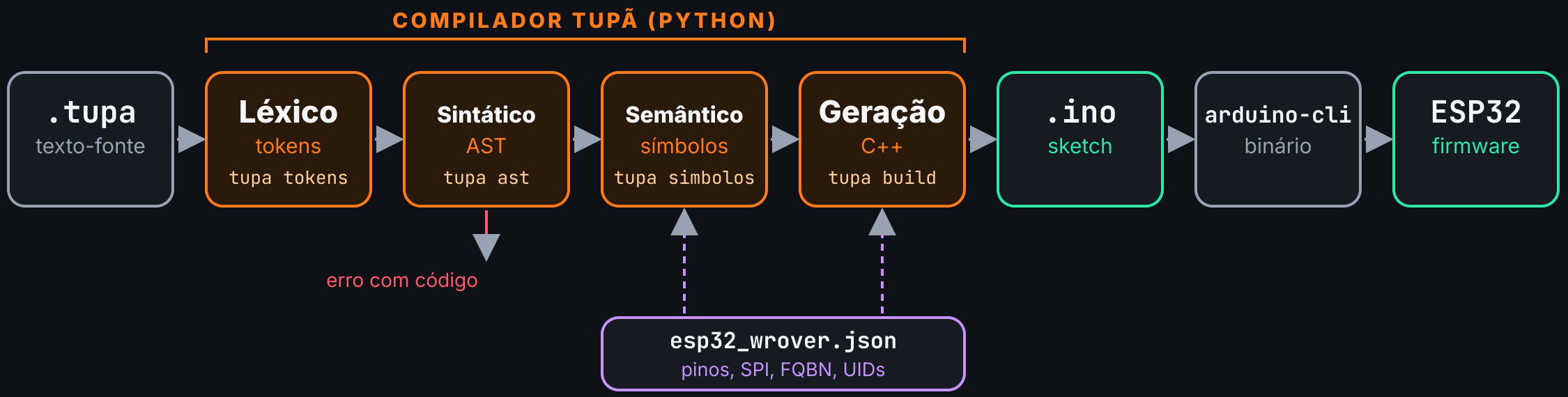
        se cartao.valido == verdadeiro {
            led.verde();
            esperar(1500);
        } senao {
            led.vermelho();
        }
    }
}
```

EVENTO	DISPARA QUANDO
`ao iniciar`	liga a placa
`ao botao(N)`	a cada aperto
`ao obstaculo`	IR detecta
`ao cartao`	cartão lido

Pinos e UIDs ficam em `esp32_wrover.json`.

ARQUITETURA

Quatro fases, depois o arduino-cli.



ANÁLISE LÉXICA

Uma regex mestre, 44 tokens.

compilador/lexer.py — regex mestre

```
_MESTRE = re.compile("|".join(
    f"(?P<{nome}>{padrao})"
    for nome, padrao in _ESPECIFICACAO))

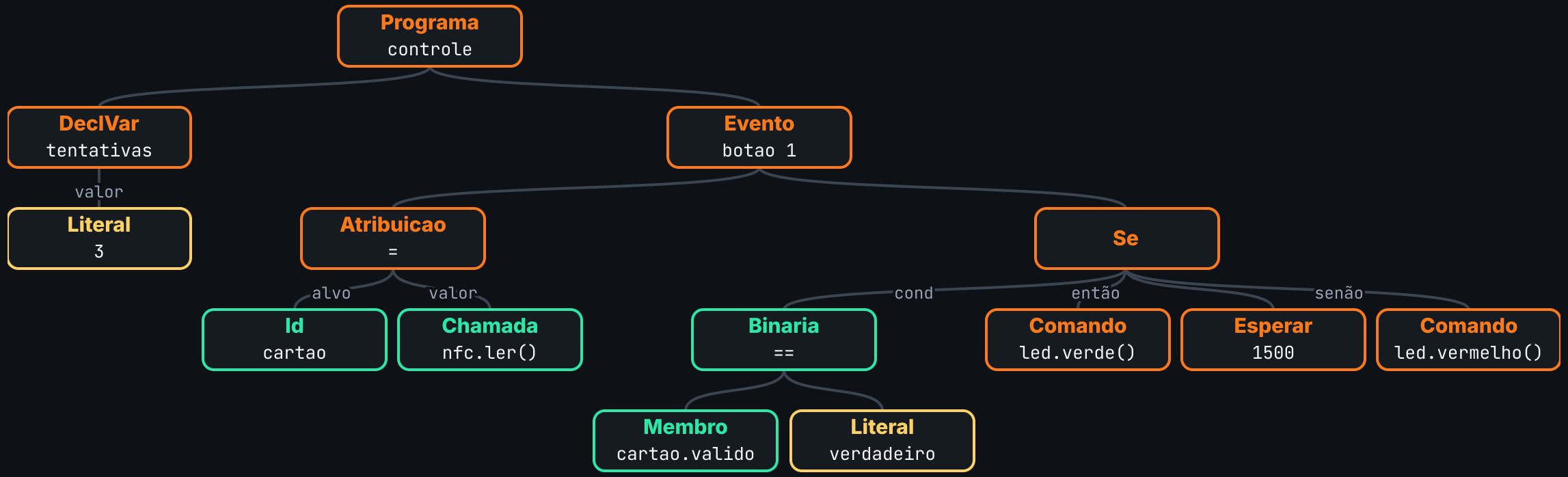
("TK_IGUAL", r"=="),   # antes de TK_ATRIB
("TK_ATRIB", r"="),
("TK_ID",    r"[a-zA-Z][a-zA-Z0-9_]*")
```

18 reservadas · ID · 3 literais · 14 operadores · 7 delimitadores · EOF

cartao = nfc.ler(); — tokenização real

LEXEMA	TOKEN	LIN:COL
cartao	TK_ID	1:1
=	TK_ATRIB	1:8
nfc	TK_NFC	1:10
.	TK_PONTO	1:13
ler	TK_ID	1:14
(	TK_ABRE_PAR	1:17
)	TK_FECHA_PAR	1:18
;	TK_PONTO_VIRGULA	1:19

Descendente recursivo, 17 nós de AST.



EBNF resumida — docs/linguagem.md

```
programa ::= "dispositivo" ID "{" item* "}" EOF
evento   ::= "ao" "botao" "(" INT ")" bloco | "ao" ID bloco
se       ::= "se" expr bloco ( "senao" bloco )?
```

precedência — uma função por regra

```
ou      ::= e ( "||" e )*
termo   ::= fator ( ("+" | "-") fator )*
posfixo ::= primario ( "." ID | "(" args? ")" )*
```

ANÁLISE SEMÂNTICA

Símbolos, tipos e a API de hardware.

```
$ tupa simbolos controle_nfc.tupa                    tabela de símbolos
```

NOME	TIPO	ESCOPO	LIN:COL
tentativas	inteiro	global	3:5
cartao	cartao	ao botao(1)	8:16

1. variável declarada antes do uso
2. tipos corretos
3. evento único
4. pino existe na placa
5. método existe no módulo

TABELA DE ERROS

40 códigos estáveis, um por condição.

1
L · léxico

13
S · sintático

21
E · semântico

5
G · placa e arquivo

Um teste por código.

exemplo.tupa — erro real

```
dispositivo d {
    ao iniciar {
        led.roxo();
    }
}
```

```
erro semantico [E19] - linha 3, coluna 12
    metodo inexistente no modulo 'led'
    disponiveis: verde(), vermelho(), apagar()
    led.roxo()

compilacao interrompida: 1 erro
```

GERAÇÃO DE CÓDIGO

Evento vira C++; a placa vem de um arquivo.

esp32_wrover.json — **arquivo de placa**

```
"pinos": { "nfc_ss": 5, "led_verde": 25,
           "botoes": { "1": 4 } },
"nfc": { "timeout_ms": 3000,
         "uids_autorizados": ["33 7A 87 F7",
                              "43 A3 6F F5"] }
```

debounce 50 ms | timeout 3 s | UIDs na placa

controle_nfc.ino, gerado — **ao botao(1) no loop()**

```
// [controle_nfc.tupa:5] ao botao(1)
  int leitura = digitalRead(BOTAO_1);
  // ... debounce por millis()
    if (botao1_estavel == LOW) {
      Serial.println("Botao: PRESSIONADO");
      Cartao cartao = nfcLer();
      if ((cartao.valido == true)) {
        ledVerde();
        delay(1500);
      } else {
        ledVermelho();
      }
      ledApagar();  // LEDs apagam ao fim do evento
```

MÉTRICAS E COMPILAÇÃO

Menos código; compila com arduino-cli.

LINHAS DE CÓDIGO	TUPÃ	C++ GERADO	RAZÃO	MANUAL
controle_nfc	12	100	8,3x	·
teste_hardware	23	112	4,9x	51

Sem linhas vazias nem comentários.

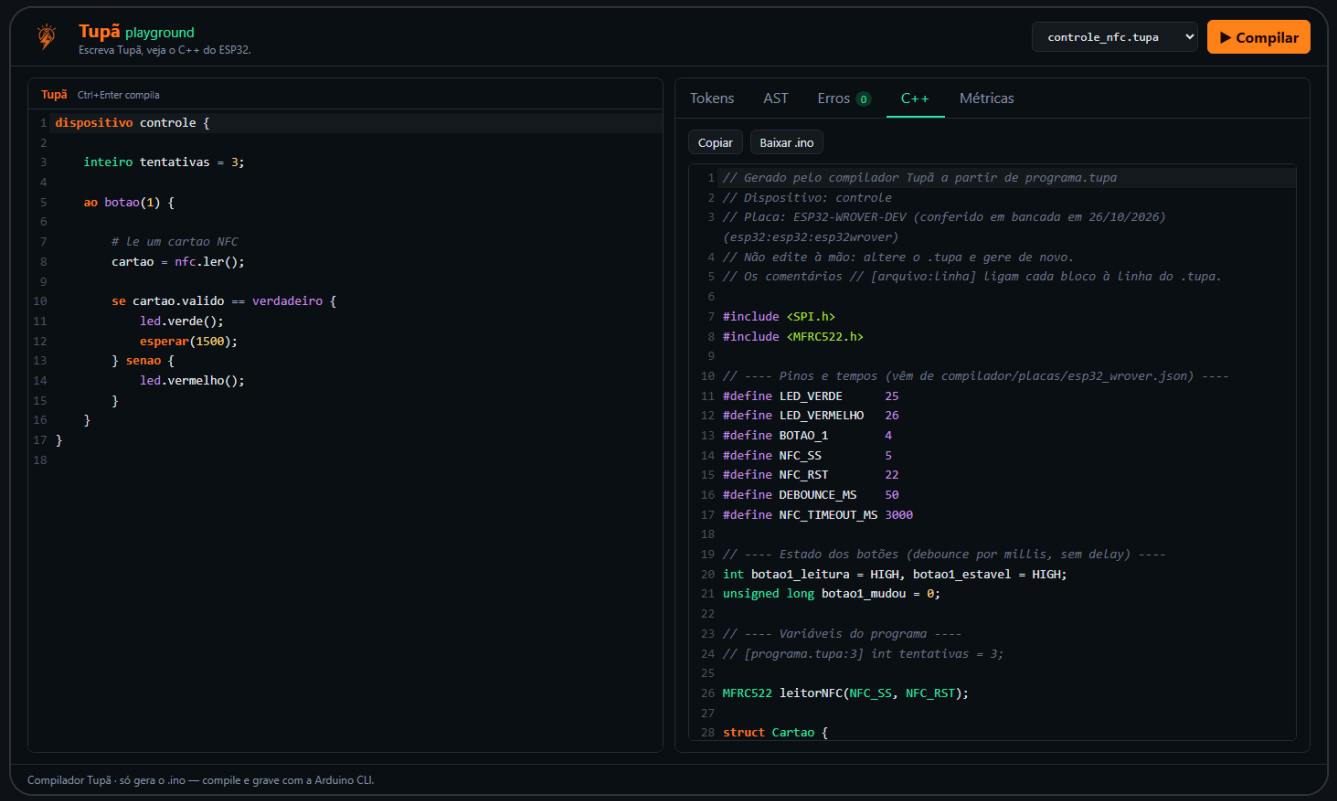
293 KB
flash · 22%

22 KB
RAM · 6%

98 testes passando

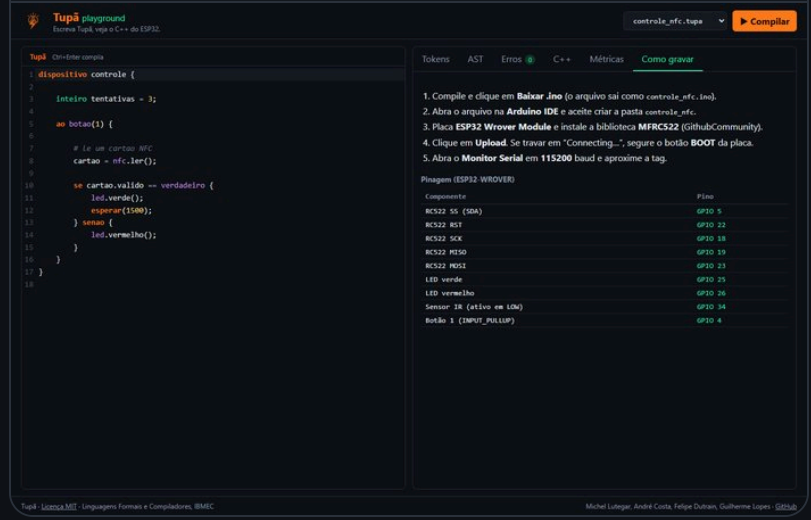
PLAYGROUND

Teste agora, sem instalar nada.

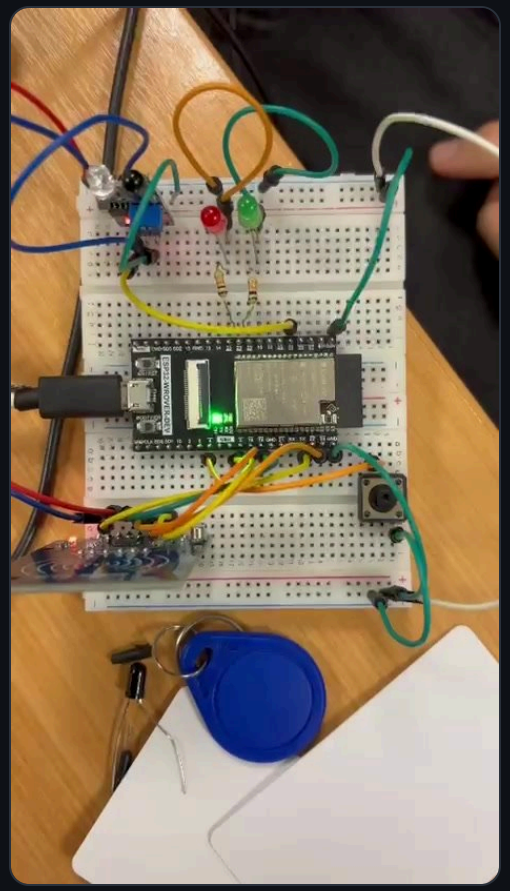


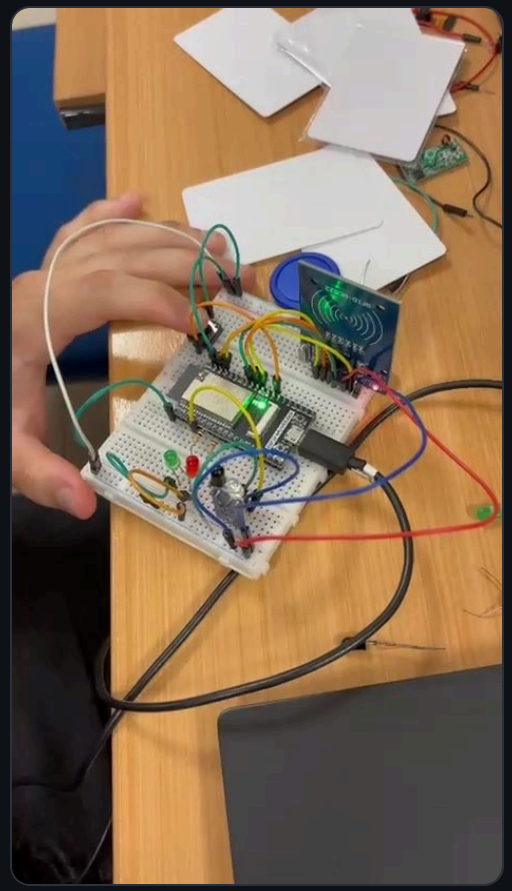
tupa.mlutegar.com

aba Como gravar



ESP32-WROVER, RC522, IR, botão e LEDs.





1. compilar
2. baixar o .ino
3. gravar

- 33 7A 87 F7 → **liberado**
- 33 8F BD F7 → **negado**
- sem cartão → **negado em 3 s**

≈ R$ 120 de material

CONCLUSÃO

Entregue, com limites claros.

Entregue

Linguagem, compilador, site e protótipo.

Limitações

Uma placa. Um erro por vez. Sem teste com usuários.

Próximos passos

Mais placas, atualização OTA, teste em sala.

Licença MIT · repositório público
github.com/mlutegar/compilador-project

Perguntas?